

# Lógica Binária

---

A **lógica binária** é a base de todo o processamento computacional. Na verdade, são estas operações mais básicas que constituem todo o poderio dos computadores. Qualquer operação, por mais complexa que pareça, é traduzida internamente pelo processador para **operações lógicas**, realizadas por meio de **portas lógicas** [1].

## Representação elétrica dos bits 0 e 1

---

Uma forma típica de representar os **bits 0 e 1** nos sistemas eletrônicos a através de níveis de tensão elétrica.

Nos **circuitos integrados (CI) TTL** (*transistor transistor logic*) utiliza-se os seguintes níveis de tensão:

□ **0V** (nível **BAIXO**) para representar o **0**;

□ **5V** (nível **ALTO**) para representar o **1**.

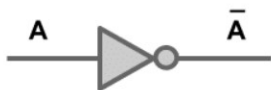
Outras tecnologias de CI, como a CMOS, utilizam outros níveis de tensão.

## Portas Lógicas

---

As **portas lógicas** implementam os operadores da **lógica binária** e são os principais componentes da eletrônica digital.

### Porta NÃO (NOT)



| A | $\bar{A}$ |
|---|-----------|
| 0 | 1         |
| 1 | 0         |

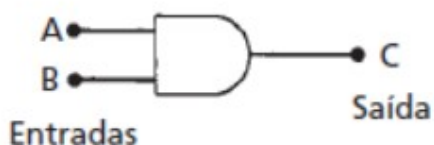
Implementa o operador **NÃO (NOT)**, ou **negação binária**, resulta no complemento do operando, ou seja, será um bit 1 se o operando for 0, e será 0 caso contrário, conforme podemos confirmar pela **tabela verdade**, onde A é o bit de entrada e S é o bit de saída (out):

| A | S |
|---|---|
| 0 | 1 |
| 1 | 0 |

Expressão booleana:

$$S = \sim A$$

## Porta E (AND)



| Entradas |   | Saída |
|----------|---|-------|
| A        | B | C     |
| 0        | 0 | 0     |
| 0        | 1 | 0     |
| 1        | 0 | 0     |
| 1        | 1 | 1     |

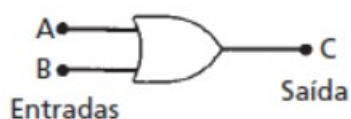
Implementa o operador **E (AND)**, ou **conjunção binária**, devolve um bit 1 sempre que ambos operandos sejam 1, conforme podemos confirmar pela **tabela verdade**, onde A e B são bits de entrada e S é o bit de saída (out):

| A | B | S |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Expressão booleana:

$$S = A \cdot B$$

## Porta OU (OR)



| Entradas |   | Saída |
|----------|---|-------|
| A        | B | C     |
| 0        | 0 | 0     |
| 0        | 1 | 1     |
| 1        | 0 | 1     |
| 1        | 1 | 1     |

Implementa o operador **OU (OR)**, ou **disjunção binária**, devolve um bit 1 sempre que pelo menos um dos operandos seja 1, conforme podemos confirmar pela **tabela verdade**, onde A e B são bits de entrada e S é o bit de saída (out):

| A | B | S |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

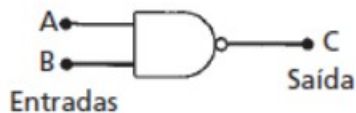
Expressão booleana:

$$S = A + B$$

### Portas Não E (NAND)

Corresponde ao operador E seguido por uma negação.

| A | B | S |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |



| Entradas |   | Saída |
|----------|---|-------|
| A        | B | C     |
| 0        | 0 | 1     |
| 0        | 1 | 1     |
| 1        | 0 | 1     |
| 1        | 1 | 0     |

Expressão Booleana

$$S = \sim(A \cdot B)$$

### Porta NÃO OU (NOR)

Corresponde ao operador OU seguido por uma negação.

| A | B | S |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |



Tabela da Verdade

| A | B | $S = (A+B)'$ |
|---|---|--------------|
| 0 | 0 | 1            |
| 0 | 1 | 0            |
| 1 | 0 | 0            |
| 1 | 1 | 0            |

Expressão Booleana

$$S = \sim(A + B)$$

### Porta OU-EXCLUSIVO (XOR)



Tabela da Verdade

| A | B | $S = (A \oplus B)$ |
|---|---|--------------------|
| 0 | 0 | 0                  |
| 0 | 1 | 1                  |
| 1 | 0 | 1                  |
| 1 | 1 | 0                  |

Implementa o operador **OU-EXCLUSIVO (XOR)**, ou **disjunção binária exclusiva**, devolve um bit 1 sempre que o número de operandos iguais a 1 é ímpar, conforme podemos confirmar pela **tabela verdade**, onde A e B são bits de entrada e S é o bit de saída (out):

| A | B | S |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

Expressão booleana:

$$S = A \oplus B$$

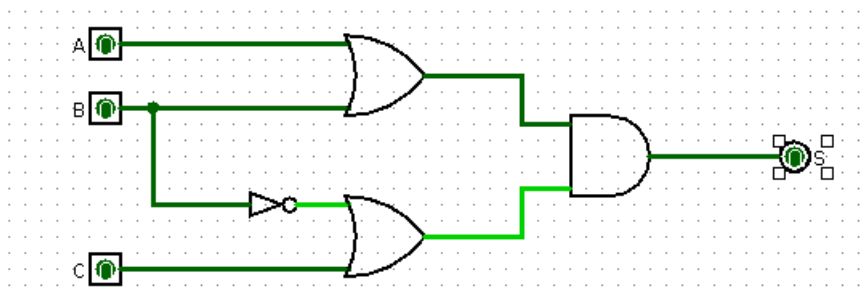
## Descrevendo circuitos lógicos algebricamente

Qualquer circuito lógico pode ser descrito usando os três operadores lógicos fundamentais: **E**, **OU** e **NÃO**.

### Exemplo

Circuito construído a partir da expressão lógica:

$$S = (A + B) \cdot (\sim B + C)$$



Note que:

- As expressões  $(A + B)$  e  $(\sim B + C)$  são entradas de uma porta E;
- A expressão  $A + B$  é gerada por portas OU;
- A expressão  $\sim B + C$  também é gerada por portas OU, com um inversor na entrada B.

### Tabela Verdade

O funcionamento de um circuito lógico é determinado através de sua **tabela verdade**.

Vejamos como construir a tabela verdade para o circuito  $S = (A + B).(\sim B + C)$ .

| A | B | C | (A+B) | $\sim B$ | ( $\sim B+C$ ) | S |
|---|---|---|-------|----------|----------------|---|
| 0 | 0 | 0 | 0     | 1        | 1              | 0 |
| 0 | 0 | 1 | 0     | 1        | 1              | 0 |
| 0 | 1 | 0 | 1     | 0        | 0              | 0 |
| 0 | 1 | 1 | 1     | 0        | 1              | 1 |
| 1 | 0 | 0 | 1     | 1        | 1              | 1 |
| 1 | 0 | 1 | 1     | 1        | 1              | 1 |
| 1 | 1 | 0 | 1     | 0        | 0              | 0 |
| 1 | 1 | 1 | 1     | 0        | 1              | 1 |

## Operadores Booleanos nas Linguagens de Programação

As **portas lógicas (E, OU e NÃO)** também são implementadas como **operadores booleanos** em linguagens de programação.

Por exemplo, na **Linguagem C**, temos os operadores:

□! (não lógico);

□&& (e lógico);

□|| (ou lógico).

Exemplos de códigos

### NÃO lógico

Este operador pode ser utilizado dentro de uma condição SE:

```
if (!x) { // se x não é verdadeiro
    // faça
}
```

Também pode ser utilizado para inverter um valor booleano.

```
x = !y; // inverte o valor de y armazenado em x
```

### E lógico

O operador E também pode ser utilizado dentro de uma condição SE:

```
if (a == 10 && b == 10) { // se ambas variáveis forem iguais a 10
    // faça
}
```

### OU lógico

O operador OU também pode ser utilizado dentro de uma condição SE:

```
if (a > 0 || b > 0) { // se uma ou outra variável for maior que 0
  // faça
}
```

## Laboratório e Exercícios de Simulação

---

Para este laboratório será utilizado o **Simulador de Circuitos Lógicos - Logisim**, o qual permite o projeto e a simulação de circuitos lógicos através de uma interface gráfica.

O *download* do **Logisim** pode ser obtido no endereço: <http://sourceforge.net/projects/circuit/>

### Circuitos com portas lógicas

Construir, simular e determinar a **tabela verdade** dos circuitos representados pelas **expressões lógicas** abaixo:

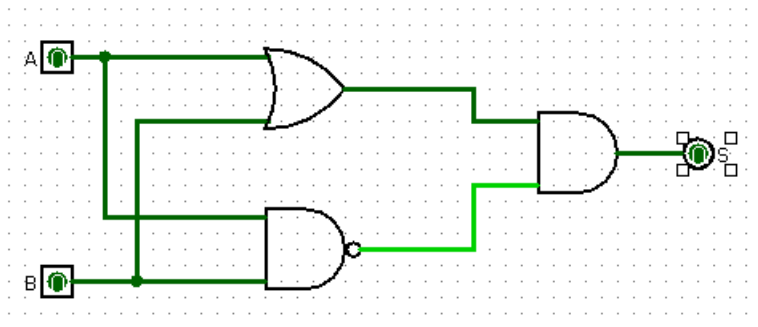
1.  $S = (A + B).(\sim B + C)$

2.  $S = A.C + \sim(B.C) + \sim A.B.C$

3.  $S = A.B + \sim C + \sim(C.D)$

4.  $Z = X.(\sim Y + W)$

5. Construir e simular o circuito da figura abaixo e comparar seu funcionamento com a porta **Ou-Exclusivo**:



# Álgebra de Boole

---

## Teoremas Booleanos

1.  $A \cdot 0 = 0$
2.  $A \cdot 1 = A$
3.  $A \cdot A = A$
4.  $A \cdot \sim A = 0$
5.  $A + 0 = A$
6.  $A + 1 = 1$
7.  $A + A = A$
8.  $A + \sim A = 1$

## Teoremas de DeMorgan

1.  $\sim(A + B) = \sim A \cdot \sim B$
2.  $\sim(A \cdot B) = \sim A + \sim B$

## Exercício

Construir e simular circuitos para comprovar experimentalmente os Teoremas de DeMorgan:

1.  $\sim(A + B) = \sim A \cdot \sim B$
2.  $\sim(A \cdot B) = \sim A + \sim B$

## Referências

---

1. ↑ TOCCI, R.J.; WIDMER, N.S.; MOSS, G.L. Sistemas Digitais: princípios e aplicações, São Paulo: Pearson, 2011.